

PlanetPulse Project Implementation and Requirements Report

Azisly Hackathon | Track 2 Web Product | Hackathon ID AZIS-W7S9Q2

Prepared 22 September 2026

Project assessment

PlanetPulse is a working implementation of a shared carbon footprint tracker, with a React browser interface, an Express API, and MongoDB storage. Its source contains the required technology stack, all seven required API routes, all six prescribed emission factors, and the Monday-to-Sunday calendar-week calculation. The application deliberately has no login or signup. Everyone uses the same activities and weekly target.

The project should not yet be described as fully compliant or ready for submission. The dashboard omits the all-time footprint even though the API calculates it. A saved weekly target of zero is shown as 25, and a failure in the optional insights request can prevent the main dashboard from loading. Other limitations include settings-document concurrency, extreme-number overflow, and incomplete deployment verification. These findings come from the actual source, not from earlier completion messages.

How the project was created

The owner supplied the product idea, hackathon constraints, and phase-by-phase prompts. Codex generated and revised the JavaScript, JSX, styles, configuration, and documentation in response. This was an AI-assisted development process: the owner directed the requirements, while Codex performed the coding and local verification. There is no evidence that the running application calls an AI model; its calculations, forecast, and tips are deterministic code.

The work proceeded from the project scaffold to MongoDB models, reusable calculation services, API controllers, dashboard aggregation, frontend routing, individual screens, validation, a visual redesign, and submission documentation. This order allowed later screens to consume an already-defined API. Section 7 describes those stages in more detail.

What this report establishes

This report explains the application in plain language, maps each tool to its role, traces the flow between the browser and database, and evaluates the written project overview and phase requirements. Source references such as [S3] point to the file index at the end. The findings distinguish functionality present in code from checks performed during this review and items that still need live deployment testing.

The fresh frontend production build passed. Targeted backend checks confirmed the factors, common validation rules, unusual-value confirmation, and calendar-week boundaries. The backend was not listening on port 5000 during this review, so these results do not certify current end-to-end database connectivity or a public deployment.

Reading guide

Sections 2 to 6 explain the tools and application behavior. Section 7 explains the development sequence and visual design. Section 8 provides the requirement-by-requirement verdict. Sections 9 and 10 describe the gaps, verification evidence, and submission work. Section 11 identifies the source files used in the analysis.

2 Tools used and their roles

The application has three main layers. The frontend is the part the visitor sees. The backend receives requests and applies the product rules. The database stores records so they survive page refreshes and server restarts. Each library supports a particular part of that arrangement. [S1-S3]

Tool	Role in PlanetPulse	How it connects
React and JavaScript	Build screens and update them when form values or API data change.	Components render inside the root element in index.html.
Vite	Runs the development server and creates the production frontend bundle.	Uses the React plugin and processes the client source.
React Router	Selects Dashboard, Log Activity, History, or Settings from the URL.	BrowserRouter wraps App and its public route definitions.
Tailwind CSS	Supplies utility classes for layout, colors, typography, and responsiveness.	PostCSS processes Tailwind and Autoprefixer into CSS.
Axios	Sends HTTP requests and receives JSON responses.	One api.js client points at VITE_API_URL or localhost port 5000.
Recharts	Draws the category donut chart and its tooltip.	Receives transport, electricity, and food totals from the dashboard API.
Node.js and Express	Run the JavaScript server and handle HTTP routes.	Express passes each API request to its controller.
Mongoose	Defines document fields and exposes database operations.	Controllers use models to query MongoDB, save activities, and aggregate totals.
MongoDB Atlas	Hosts the persistent MongoDB database used during development.	Mongoose connects using the server-only MONGODB_URI.
cors and dotenv	Permit browser requests across origins and load local server configuration.	Installed as server middleware and startup configuration.
Nodemon and pnpm	Restart the development server and manage dependencies or scripts.	Nodemon is for development; production start runs node server.js.

Development tools and installed versions

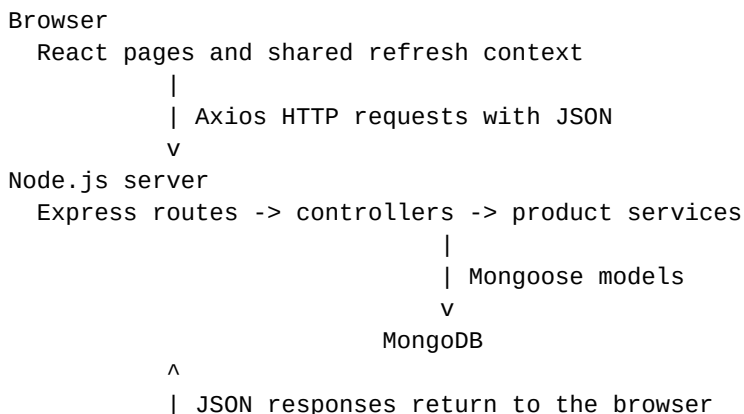
The development workflow used Codex for implementation and review, PowerShell for local commands, and browser inspection for earlier UI checks. VS Code is the owner's available editor; it is not an application runtime dependency. Google Fonts supplies Fraunces and Inter for display text and interface text. No image-generation service is required by the app.

The installed packages inspected include Vite 8.3.0, Express 5.2.1, Mongoose 9.10.1, cors 2.8.6, dotenv 18.0.1, and Nodemon 3.1.14. These are local installed versions, not a claim about the latest public releases. Most package declarations use "latest"; separate pnpm lockfiles record resolved versions. [S1]

There is a documentation mismatch: README says Node 18 or later, but installed Vite requires Node ^20.19.0 or >=22.12.0, and Mongoose requires >=20.19.0. The report recommends a runtime meeting both installed engine requirements and an updated README. No TypeScript, Next.js, Firebase, Supabase, PostgreSQL, Python backend, Django, or Laravel is used in the inspected application source.

3 How the layers work together

The integration follows a request-and-response flow. A person changes a React form; Axios sends JSON to Express; a controller validates the request, calls a service, and uses Mongoose to access MongoDB. Express returns JSON, and React displays the result. The browser never receives the database connection credentials. [S2-S6]



A normal activity from input to saved record

Suppose a visitor selects Car and enters 12. The form switches the unit to km and previews 2.4 kg of CO₂ using the car factor of 0.20. The visitor submits the form, which sends only the type and quantity needed for calculation. Client-side validation first checks that the quantity is present, numeric, finite, and greater than zero. [S4]

POST /api/activities reaches the activity controller. The backend validates the input again, looks up the car category and factor, and calculates the final emissions. It saves an Activity document containing the original quantity, transport category, km unit, factor 0.20, CO₂ value 2.4, unusual flag, and creation time. A successful response uses HTTP 201 and returns the saved record. Client-supplied category, factor, or CO₂ values do not control this calculation. [S5-S6]

The form then clears and displays the server-returned result. It increments dataVersion in ActivityDataContext. Dashboard and History observe that value and reload their data when needed.

Settings updates use the same refresh mechanism. This connects the screens within one running browser session; it is not automatic synchronization across different visitors' browser tabs. [S2]

An unusual activity

For a 500,000 km car journey, the server initially returns HTTP 200 with requiresConfirmation set to true and does not save the record. The frontend opens a confirmation dialog. Cancel returns to editing; Confirm and Log sends the original quantity again with confirmUnusual set to true. The backend saves 100,000 kg of CO2 and marks flaggedUnusual as true. This preserves the person's choice while making the implausible input visible. [S4-S6]

Startup and the database dependency

The server loads its environment configuration and waits for mongoose.connect before opening its HTTP port. Therefore, a database connection failure also means the API health route is unavailable. The frontend can still start independently and show friendly request errors. Earlier startup logs reported inability to reach the Atlas cluster; an IP allowlist issue was suggested by that generic error, but it was not conclusively isolated from other network or cluster causes. [S3]

4 Data storage and calculation rules

An Activity document represents one logged choice. A Settings document holds the shared target. There is no user ID, account ownership, password, or token field because the brief requires a public shared dataset. MongoDB stores dates, while the frontend formats them for display. [S7]

Record	Fields	Purpose
Activity	type, category, quantity, unit	Identifies the activity and the amount entered.
Activity	emissionFactor, co2	Stores the factor used and the server-calculated emissions.
Activity	flaggedUnusual, createdAt	Records confirmed unusual input and when it was logged.
Settings	weeklyTarget, updatedAt	Stores the shared target and its latest update time.

Fixed emission factors

The authoritative server rule is multiplication: CO2 in kilograms equals quantity multiplied by the prescribed factor. The frontend repeats the factors only to display a preview. Ordinary floating-point calculation is stored without forced display rounding; number formatting on the screens limits the visible decimal places. [S6]

Type	Category	Unit	Factor in kg CO2 per unit
car	Transport	km	0.20
bus	Transport	km	0.08
flight	Transport	km	0.25

Type	Category	Unit	Factor in kg CO2 per unit
electricity	Electricity	kWh	0.80
veg_meal	Food	meals	0.50
non_veg_meal	Food	meals	2.00

Validation and unusual thresholds

Unsupported activity types, nonnumeric quantities, nonfinite quantities, zero, and negative quantities receive validation errors in the normal API flow. Unusual thresholds are greater than 1,000 km for car or bus, 15,000 km for flight, 200 kWh for electricity, and 10 meals for either meal type. Equality with a threshold is not unusual. [S6]

The frontend displays and handles the backend's unusual warning, but it does not independently implement these thresholds. That distinction matters when judging the original requirement for checks on both frontend and backend. Extreme numeric overflow also remains possible after confirmation, as described in Section 9.

Shared settings and persistent history

The controllers use the first Settings document, creating a default target of 25 kg if none exists. This is intended to behave as one shared record, but the database does not enforce a singleton key. Concurrent first requests can both create a record. Historical activities remain stored when a new week begins; weekly queries simply select a different date range. Deletion occurs only through the explicit activity-delete API. [S5, S7-S9]

5 API contract and responsibilities

The REST API uses HTTP methods and URL paths to identify each action. JSON carries the request and response data. All seven required method-and-path combinations are present, together with the additive insights endpoint. No authentication is applied to these routes. This is source-level confirmation; current live database execution was not re-certified. [S3, S5, S8, S9]

Method and path	Implemented behavior
POST /api/activities	Validates type and quantity, handles unusual confirmation, calculates emissions, and saves an activity.
GET /api/activities	Returns newest-first records. Supports category, from, and to query parameters.
GET /api/activities/weekly	Returns current-week records and their totalCO2.
DELETE /api/activities/:id	Validates the ID, deletes the matching record, and returns a message and the deleted activity.
GET /api/settings	Returns shared settings, creating the 25 kg default if necessary.

Method and path	Implemented behavior
PUT /api/settings	Accepts a finite, non-negative weeklyTarget and persists it.
GET /api/dashboard	Returns all-time totalCO2, weeklyCO2, weeklyTarget, remaining, percentUsed, categoryBreakdown, and exceeded.
GET /api/dashboard/insights	Returns equivalence comparisons, forecastKg, topCategory, and a constructive tip.

Queries and error responses

History category values are transport, electricity, and food; omitting category means all. The frontend converts selected local dates into inclusive ISO timestamps for the from and to query parameters. The backend rejects unsupported categories, invalid dates, and reversed ranges. It sorts results by createdAt descending. [S5, S11]

The activity controller returns HTTP 400 for invalid inputs, HTTP 200 for an unusual-value warning awaiting confirmation, and HTTP 201 after saving a new record. Delete returns 400 for a malformed ID and 404 for a missing record. Shared error middleware handles malformed JSON, Mongoose errors, and unexpected failures with readable messages. [S5, S10]

Frontend connection and production configuration

The shared Axios instance uses VITE_API_URL, falling back to http://localhost:5000. Vite also defines a development /api proxy, but the default absolute Axios base URL sends requests directly to the API and bypasses that proxy. Express CORS middleware permits cross-origin browser access. A deployed frontend therefore needs the correct public API URL at build time. [S1-S3]

The application has no export endpoint or export button. The architecture reference mentioned an optional /api/activities/export route, but the selected differentiation features were equivalences, forecast, and a category-based tip. Export is an unimplemented optional item, not one of the seven graded routes. The DELETE endpoint exists even though the current History screen has no delete button.

6 Screens and product decisions

Dashboard

The dashboard requests summary data, insights, and activities. It displays this week's CO2, target, remaining allowance, percentage used, a progress bar, a category donut, the latest five activities, equivalences, and a forecast. The recent-activity list is obtained by downloading the history and taking its first five entries. The donut legend provides readable category values alongside the chart. The all-time total returned by the API is not rendered. [S8, S12]

Log Activity

The form provides six activity types and changes the unit between km, kWh, and meals. It shows a live CO2 preview, validates the quantity, reports success or error, and provides confirm/cancel controls for unusual input. The backend remains authoritative. The confirmation dialog has accessible labels, but there is no implemented focus trap, focus restoration, or Escape-key handler, so complete keyboard accessibility should not be claimed. [S4]

History and Settings

History shows activity type, quantity, unit, emissions, and date. It has category and date filters, retry feedback, and distinct empty states for no records and no matching records. The table can scroll horizontally on narrow screens. Settings loads and saves the shared target, then shows a toast and refreshes dependent screens. Blank target input currently converts to zero, which is accepted; this is a validation gap. [S11, S13]

DP1 The Nudge

When exceeded is true, NudgeBanner displays the overage, target comparison, and a practical tip. It contains no mechanism that blocks logging and uses constructive language. The intended behavior is present, although the zero-target display bug can make the stated overage incorrect. The category tip is visible only while the target is exceeded, even though the API supplies it at other times. [S12]

DP2 Absurd Input

The warning-and-confirmation flow preserves unusual input instead of silently modifying it. The backend decides whether the quantity crosses a threshold; the frontend displays the warning and allows correction or confirmation. This satisfies the visible confirmation flow, but independent client-side threshold checking and complete overflow protection are still absent. [S4-S6]

DP3 The Week

getCurrentWeekRange finds Monday at 00:00:00.000 and Sunday at 23:59:59.999 in the server's local timezone. The controllers use inclusive date bounds. The interface explains Monday-Sunday, and no reset job deletes old records. A production host's timezone must be chosen deliberately because server-local time can differ from a visitor's local date. [S8-S9]

7 Development sequence and design

The phase sequence explains how the pieces were assembled. These stages summarize the briefs and the resulting files; they do not imply that every acceptance condition passed merely because a phase was completed.

Phase	Work performed	Result in the project
1 to 2	Scaffold client/server and define database models.	Package scripts, environment examples, Activity and Settings schemas.
3	Isolate emission, week, and validation rules.	Services reused by API controllers.

Phase	Work performed	Result in the project
4 to 5	Implement required APIs and dashboard aggregation.	Activity/settings controllers, summary and insights endpoints.
6	Establish frontend routing and shared API access.	App routes, sidebar, Axios wrappers.
7 to 10	Build logging, dashboard, history, and settings screens.	Forms, chart, filters, confirmation dialog, progress and feedback.
11	Add validation and error-state polish.	Positive quantity checks, retry controls, readable failures, responsive review.
11b	Apply a visual redesign.	Shared color tokens, Fraunces/Inter, active indicators, factual weekly headline.
12	Write project documentation and deployment instructions.	Root README and DECISIONS with the real hackathon ID.

Visual design integration

CSS variables define sage background #F2F4EF, ink #1C2620, pine primary #2F5233, amber accent #C98A3B, muted warning #B54834, and border #D8DED2. Tailwind maps these variables to reusable pp utility classes. Fraunces is used for headings and prominent numbers, while Inter supports labels, navigation, and body text. [S14]

The dashboard replaced the static greeting with a weekly status derived from available data. A pine-filled total card carries more emphasis than secondary bordered panels. Mobile navigation uses an active underline; desktop navigation uses a left-edge indicator. The forms and History screen share the same typography and surface treatment. Most decorative gradients and card shadows were removed.

The chart still stores three hard-coded color values instead of referring directly to the shared CSS variables. This is a minor maintainability mismatch with the redesign brief, not a functional API defect. Google Fonts also requires network access; fallback serif and system fonts are defined for unavailable font downloads.

Additive insights

The insights controller converts the current weekly total into tree-days using 21 kg per tree per year, comparable car kilometres using 0.20 kg per kilometre, and smartphone charges using 0.008 kg per charge. These are illustrative comparisons, not measurements of actual trees planted, travel avoided, or carbon offset. The UI labels them as estimates. [S8]

Forecast divides weekly emissions by elapsed calendar-day count, including today, then multiplies by seven. It is a simple constant-pace projection, not machine learning. The category with the largest weekly total selects a predefined constructive tip. These outputs do not change the prescribed emission factors, but their frontend request must be isolated from the required dashboard request to ensure optional failures cannot disrupt the core screen.

8 Requirement verification

The verdicts below compare the current source against the original overview and phase briefs. Implemented means the code path exists and matches the ordinary rule; it does not replace live deployment acceptance testing. Partial means a stated requirement or supported edge case remains incomplete. Pending means the evidence needed for completion is not established.

Condition	Verdict	Evidence and qualification
Exact required technology stack	Implemented	React/Vite/JavaScript, Tailwind, Router, Axios, Recharts, Express, MongoDB/Mongoose are present. [S1]
No authentication	Implemented	All routes are public; no account, JWT, password, or user model. Public URL usability still needs deployment testing. [S2-S3]
Log six types with correct units	Implemented	All six selections and their km/kWh/meals units are in the form and server map. [S4, S6]
Server-authoritative fixed factors	Implemented	All six factors match; targeted normal-value checks passed. Overflow remains an edge gap. [S6]
Complete required dashboard	Partial	Weekly metrics/chart exist, but the all-time total is omitted and zero target displays as 25. [S12]
Persist shared weekly target	Partial	Settings GET/PUT and UI exist; database singleton is not enforced and zero display is wrong. [S7, S9, S13]
Filterable newest-first history	Implemented	Category/date filters, required columns, server sorting, and empty/error states are present. [S5, S11]
Seven required API routes	Implemented	Exact method/path combinations exist. Full live integration needs a connected database. [S3, S5, S8, S9]
DP1 friendly non-blocking nudge	Implemented with caveat	Constructive tip and no blocking behavior; zero-target bug can misstate overage. [S12]
DP2 unusual input on both sides	Partial	Server thresholds plus client confirmation exist; no independent client thresholds or full overflow guard. [S4-S6]
DP3 Monday-Sunday and retention	Implemented	Calendar boundaries and historical retention are implemented; timezone is server-local. [S8-S9]
Additive insights do not disrupt core	Partial	Separate endpoint exists, but Promise.all makes dashboard loading depend on optional insights. [S8, S12]
README and product decisions	Implemented with correction needed	ID and DP1/2/3 are documented; Node prerequisite and all-time UI claim need correction. [S15]
Production and submission readiness	Pending	Build passes; live backend, public GitHub repo, deployed URL, and demo recording are not verified. [S16]

The optional export endpoint is absent. It should be described as future work unless explicitly selected for implementation. A project-wide pass score would hide the partial items above, so this report does not assign an unsupported percentage of compliance.

9 Changes needed before claiming full compliance

Required fixes

Display the all-time footprint. The server returns totalCO2 separately from weeklyCO2, but DashboardPage does not consume it. Add a clearly labelled all-time figure while retaining the weekly cards. Verify with an activity from a previous week so the two values visibly differ. [S8, S12]

Respect a target of zero. The expression `dashboard?.weeklyTarget || 25` treats zero as missing. Use a fallback that applies only to absent data, and check the headline, target card, progress text, and NudgeBanner at zero. The backend uses a special percentage convention for a zero target: zero percent for no emissions and 100 percent when emissions exist. That convention should be explained consistently. [S8, S12]

Keep optional insights failures separate. DashboardPage loads summary, insights, and history in one Promise.all. If any request fails, the successful summary is not applied. Handle these requests independently so summary data remains available when an optional section cannot load. This directly addresses the additive-features requirement. [S12]

Complete unusual-input handling. Preserve the existing confirmation flow, add the required client threshold warning, and verify that calculated CO2 remains finite after confirmation. A finite but extreme non-veg quantity can multiply to Infinity, pass the in-memory model validation tested here, and serialize as null. Return a clear correction request for an unrepresentable result. [S4-S7]

Data consistency and input quality

The intended single Settings document needs a database-enforced identifier and an atomic create-or-update path. The present findOne-then-create sequence can race when an empty database receives parallel requests. Also reject an empty weekly-target field explicitly: `Number("")` currently converts it to zero without telling the visitor that a blank entry became a numeric value. [S7, S9, S13]

Dashboard loading and initial errors should show unavailable or loading values instead of plausible zero totals and a default target. Context refresh currently updates only the active application instance; different visitors need a refresh or navigation to see others' changes. Full multi-user live updates are not required by the brief, but should not be claimed. [S2, S12]

Documentation and deployment quality

Correct the Node prerequisite to match installed dependencies, and avoid presenting the all-time value as visible until the UI is fixed. Select a deployment timezone, configure HTTPS frontend/API URLs, and add a single-page application fallback so opening `/history` or `/settings` directly loads `index.html`. Use the lockfiles for reproducible installs and preserve server-only credentials. [S1, S15]

For future scale, paginate activity history, fetch recent activities directly instead of loading all records, and consider database indexes and aggregation for large datasets. These are improvements beyond a small shared hackathon demo. None of the fixes listed in this section was applied as part of this analysis report.

10 Verification and submission readiness

Checks performed during this review

The client production command ran successfully through the configured build script. Vite transformed 666 modules and produced the dist output. The JavaScript bundle was 660.70 kB, or 202.65 kB compressed, and Vite issued a non-blocking warning for a chunk larger than 500 kB. Build success demonstrates that the frontend can be compiled; it does not prove API availability or all user flows.

Targeted backend checks used local services and an in-memory controller stub. They verified all six factors with quantity 10; rejection of zero, negative, null, string, NaN, and Infinity values; flagging of a 500,000 km car trip; and preservation of that quantity after confirmation. The stub confirmed that warning responses avoid saving and that supplied client CO2/factor values cannot override the server calculation.

Week-boundary checks covered Sunday, Monday at midnight, and a year-crossing week. These passed. The audit also reproduced the extreme-number overflow gap without writing to MongoDB. The project does not contain a configured automated test suite, so these targeted checks should not be described as comprehensive regression coverage.

Current runtime status

Port 5173 was listening during the review; no service was listening on port 5000. Earlier development sessions demonstrated live activity logging and dashboard updates, but current Atlas connectivity and a clean production-server startup were not established in this review. The previous Atlas connection error identified a connection problem, with allowlisting as one possible cause rather than a proven diagnosis.

Checks still required for submission

Item	Present assessment	Completion evidence
Core features end to end	Partial	Fix the reported gaps, then log and retrieve data against MongoDB on every required route.
Public access without auth	Code meets rule	Open the deployed URL in a fresh session and use each screen.
DP1 DP2 and DP3 visible	Mostly present	Demonstrate overage, unusual confirm/cancel, and week-boundary behavior after fixes.
Additive features isolated	Partial	Make insights fail deliberately and confirm core dashboard still loads.
Root README hackathon ID	Present	AZIS-W7S9Q2 appears in the root README.
Public GitHub repository	Not established	The inspected directory is not a Git repository; verify a public repository URL.
Deployed URL and recording	Not established	Provide the public app URL and a recording of the required flows.

Suggested demonstration sequence

Open the dashboard without signing in, log a normal activity, show the updated calculation and chart, filter the History table, update the weekly target, and trigger a friendly overage nudge. Enter 500,000 km to show the confirmation dialog and cancel it to avoid distorting the shared demo. Explain the Monday-Sunday rule, retained history, forecast, and estimate assumptions. Record the final demonstration only after database connectivity and the required fixes are verified.

11 Source evidence and terminology

All source paths below are relative to D:\PlanetPulse unless a phase-document path is specified. Line numbers refer to the files inspected for this report. References support code observations; targeted test results and runtime limitations are described separately in Section 10.

Ref	Source location	Evidence used
S1	client/package.json; server/package.json; client/vite.config.js; client/postcss.config.js	Stack, scripts, build configuration; installed package manifests for versions and engines.
S2	client/src/main.jsx:7; App.jsx:22; context/ActivityDataContext.jsx:5; services/api.js:3	React startup, routing, dataVersion refresh, Axios base URL and wrappers.
S3	server/server.js:1 and :29; server/routes/*.js	Middleware, route mounting, database-before-listen startup.
S4	client/src/pages/LogActivityPage.jsx:6, :37, :52 and :82	Activity options, client validation, warning handling, confirmation retry.
S5	server/controllers/activityController.js:13, :51, :82, :92 and :106	Filter validation, creation, sorting, weekly records, deletion.
S6	server/services/emissionService.js:1; validationService.js:3 and :12	Prescribed factors, multiplication, required checks, unusual thresholds.
S7	server/models/Activity.js:3; Settings.js:3	Stored fields and absence of enforced singleton settings key.
S8	server/controllers/dashboardController.js:21, :61, :75 and :103	Settings access, current-week aggregation, all-time total, insights.
S9	server/controllers/settingsController.js:9 and :23; services/weekService.js:1	Shared-target reads/writes and Monday-Sunday boundaries.
S10	server/middleware/errorHandler.js:7	Friendly errors and malformed JSON handling.
S11	client/src/pages/HistoryPage.jsx:40, :62 and :130; components/FilterBar.jsx:1	Date filtering, loading/error/empty states, table and category filters.
S12	client/src/pages/DashboardPage.jsx:96, :115 and :138; components/NudgeBanner.jsx; CategoryDonutChart.jsx; EquivalenceCard.jsx	Request coupling, zero fallback, cards, nudge, chart, comparisons and forecast.
S13	client/src/pages/SettingsPage.jsx:18, :45 and :57; components/Toast.jsx	Target loading, blank conversion, saving, and feedback.

Ref	Source location	Evidence used
S14	client/src/index.css:5; client/tailwind.config.js:3; client/index.html:7; components/Sidebar.jsx	Design tokens, font links, navigation and responsiveness.
S15	README.md; DECISIONS.md	Setup claims, hackathon identity, API documentation, DP1/2/3 rationale.
S16	D:\ppppppp\13-submission-checklist.md; phase briefs and project overview	Requirements and external submission deliverables.

API means the set of URLs through which the browser talks to the server. JSON is the data format used in those messages. A schema describes database fields. An endpoint is one HTTP method and URL combination. Aggregation means combining stored records into totals. A production build is the generated frontend output prepared for hosting. These terms describe the implementation; using the application does not require the visitor to know them.